Defending GenAI Applications

GenAI security handbook, volume 2 · Version 2026.1

> Sample foundation document written for testing a domain-expert RAG. It summarises public frameworks and common practice; check the primary sources (OWASP, MITRE, NIST) for current versions.

1. Defence in depth

No single control stops prompt injection. The goal is to make attacks unlikely to succeed and harmless when they do.

Layer	Control	What it stops
Ingestion	Detect hidden text and instruction-like phrases; quarantine them	Planted instructions in documents
Retrieval	Enforce access control inside the vector query	Leakage of documents the user may not read
Prompt	Spotlighting: delimit untrusted content with random boundaries and say it is data	Many naive injections
Model	Models trained to follow the instruction hierarchy	Some injections and jailbreaks
Tools	Least privilege, user-scoped credentials, allow-lists, approvals	Injections turning into actions
Output	Check for links, emails, code or actions the user did not ask for	Exfiltration and phishing
Monitoring	Log prompts, retrieved passages, tool calls and flags	Undetected abuse; supports incident response

2. Spotlighting and delimiting

Spotlighting marks untrusted content so the model can tell it apart from instructions. Techniques include delimiting (wrap each passage in tags with a random per-request token so an attacker cannot close the tag early), datamarking (interleave a marker character through the untrusted text) and encoding (for example Base64, for models

strong enough to decode it). Pair it with a system instruction that content inside the markers is data and must never be followed.

```
<source id="S2" nonce="7f3a91c2">
  ...retrieved text...
</source nonce="7f3a91c2">
```

3. Privilege separation and the dual-model pattern

Keep the model that reads untrusted content away from privileged tools. In the dual-LLM pattern a quarantined model processes untrusted text and returns only constrained, structured output (such as a category or an extracted field) to a privileged model or to deterministic code that decides actions. Capability-based designs go further and track which data came from untrusted sources so it can never flow into sensitive tool arguments without approval.

4. Securing tools and agents

- Run every tool with the permissions of the requesting user, never with a shared admin identity.
- Allow-list hosts for outbound requests; block access to cloud metadata endpoints.
- Require human approval for payments, deletions, external messages and permission changes.
- Limit steps, tokens and spend per task; stop and report when a limit is hit.
- Validate tool arguments against schemas and business rules in code, not in the prompt.
- Show the user which tool is about to run and with what arguments.

5. Secure RAG

Risk	Control
User sees restricted content	Filter by the user's groups inside the vector search, so restricted chunks never reach the model
Poisoned documents	Track document provenance, restrict who can publish into indexed sources, quarantine hidden text

Hallucinated citations	Validate that every citation refers to a passage that was actually retrieved
Stale or conflicting content	Metadata for version and date; prefer the newest authoritative source
Cross-tenant leakage	Separate collections or strict tenant filters, tested in CI

6. Output handling

Treat model output like user input to every downstream system. Escape it before rendering as HTML, never execute it as SQL or shell commands, parse structured output against a schema, and strip or confirm URLs and email addresses that did not come from the question or a trusted source. Rendering Markdown images from model output is a known exfiltration channel, because the image URL can carry data in its query string.

7. Secrets and data

Never put API keys, passwords or authorisation rules in system prompts; assume the system prompt will leak. Keep secrets in a secrets manager and inject them into tool code, not into model context. Minimise personal data in prompts, mask it where possible, and make sure model providers do not train on your data and keep it in approved regions.

8. Guardrail services

Classifier-based guardrails screen inputs and outputs for jailbreaks, prompt attacks, toxic content, personal data and off-topic requests. They are valuable as one layer but produce false positives and negatives; measure them on your own traffic and do not rely on them as the only control for high-impact actions.

9. Incident response for AI features

1. Detect: alerts on quarantine hits, output-check flags, unusual tool calls or cost spikes.
2. Contain: disable the affected tool or document source; switch the feature to read-only.
3. Investigate: use the audit log to find the prompt, passages and tool calls involved.

4. Eradicate: remove poisoned content, rotate exposed secrets, patch the control that failed.
5. Recover and learn: add the attack to the red-team suite so it is tested on every release.